MANUAL DE USUARIO — TACTICALGRID

1. Portada e Información General

Campo	Detalle
Proyecto	**TacticalGrid** — Entorno Táctico 2D con Búsqueda y Toma de Decisiones
Asignatura	Sistemas Inteligentes I
Programa	Ingeniería de Sistemas y Computación — Universidad de Caldas
Versión del Manual	1.1 (Octubre 2026) — actualizado con Beam Search y la comparación Minimax vs Alfa-Beta
Integrantes del Equipo	• Sophia Cardona • Danna Alexandra Madrid Roa • Michel Dahiana Arenas López
Tipo de Documento	Manual de Usuario Final y Guía de Experimentación

2. Tabla de Contenido

1. Portada e Información General
2. Tabla de Contenido
3. Descripción General de TacticalGrid
4. Requisitos del Sistema e Instalación
5. Estructura Visual de la Interfaz
6. Modos de Operación (Modo JSON vs. Modo Exploración)
7. Guía de Uso Paso a Paso
    - 7.1 Carga y Validación de Escenarios JSON
    - 7.2 Misión 1: Búsqueda de Rutas (Navegación Unipersonal)
    - 7.3 Misión 2: Partida Táctica por Turnos (Humano vs. IA / IA vs. IA)
    - 7.4 Misión 3: Toma de Decisiones Adversarial (Minimax y Alfa-Beta)
8. Interpretación de Pestañas y Paneles
    - 8.1 Panel de Resultados y Verificación
    - 8.2 Panel de Estadísticas y Comparación de Heurísticas
    - 8.3 Panel de Comparación de Algoritmos
    - 8.4 Panel de Escenario y Registro de Eventos (Logs)
9. Guía para Reproducir los Casos Mínimos de Evaluación
10. Escenarios Robustos de Evaluación
11. Escenarios Inválidos (Rechazo Atómico)
12. Preguntas Frecuentes y Solución de Problemas

3. Descripción General de TacticalGrid

TacticalGrid es una plataforma interactiva diseñada para observar, comparar y evaluar formalmente el comportamiento de distintos algoritmos de Inteligencia Artificial:

- **Búsqueda no informada:** Búsqueda en Anchura (**BFS**) y Búsqueda en Profundidad (**DFS**).
- **Búsqueda basada en costos:** Búsqueda de Costo Uniforme (**UCS** / Dijkstra), sensible a terrenos de distinto costo.
- **Búsqueda informada:** Algoritmo **A*** con múltiples funciones heurísticas (Distancia Manhattan, Heurística Propia informada por la meta y Distancia Euclidiana).
- **Búsqueda con memoria acotada: Beam Search** (búsqueda en haz) con un ancho de haz configurable k, que limita la frontera a los k mejores estados de cada nivel.
- **Toma de decisiones adversarial:** Algoritmo **Minimax** y Minimax con **Poda Alfa-Beta**, orientados por una función de evaluación táctica para juegos de suma cero.

El objetivo central del juego táctico consiste en dos bandos (**A** y **B**) que compiten sobre una cuadrícula con diversos terrenos y obstáculos. En el centro neutral yace un **Recurso Estratégico**; el bando que logre capturar el recurso y llevarlo hasta su respectiva base gana la partida.

4. Requisitos del Sistema e Instalación

4.1 Requisitos de Software

- **Sistema Operativo:** Windows 10/11, GNU/Linux o macOS.
- **Intérprete:** Python 3.9 o superior (probado y certificado en Python 3.11, 3.12 y 3.14).
- **Bibliotecas requeridas:** Únicamente la biblioteca estándar de Python y el soporte gráfico de `tkinter`.

> **Note**
>
> En Windows y macOS, `tkinter` viene preinstalado con el instalador oficial de Python. En distribuciones basadas en Debian/Ubuntu, si no está disponible, se instala ejecutando:

```
sudo apt update && sudo apt install python3-tk
```

4.2 Instrucciones de Ejecución

1. Abra una terminal o consola de comandos (PowerShell, CMD o Bash).
2. Diríjase a la carpeta del proyecto:

   ```
   cd "TacticalGrid"
   ```

3. Ejecute la aplicación principal:

   ```
   python main.py
   ```

   (o `python3 main.py` en sistemas Unix/Linux).
4. *(Opcional)* Para verificar la integridad de todos los módulos mediante la suite de pruebas unitarias (25 pruebas):

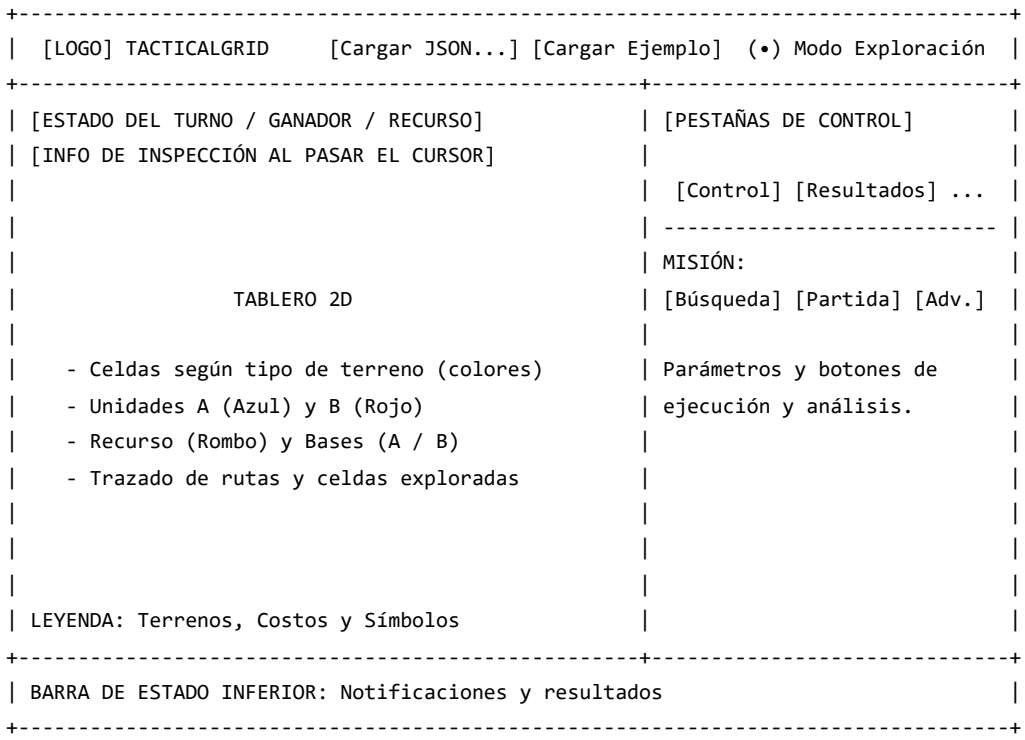
```
python -m unittest discover -s tests -p "test_*.py" -v
```

5. *(Opcional)* Para regenerar el reporte de resultados experimentales (`documentacion/resultados_experimentos_busqueda.md` , casos 1–5, robustos `R01-R08` y adversariales 7–10):

```
python experimentos/ejecutar_experimentos_busqueda.py
```

5. Estructura Visual de la Interfaz

La ventana principal de TacticalGrid se divide en dos secciones principales:



Componentes Clave:

1. **Barra Superior:** Contiene los accesos rápidos para cargar escenarios (`Cargar JSON...` , `Cargar ejemplo`), el nombre del archivo activo y el selector de modo de ejecución.
2. **Tablero Táctico (Izquierda):** Cuadrícula bidimensional interactiva que renderiza el mapa, la niebla/terrenos, las unidades, las bases y las animaciones visuales de caminos y estados explorados.
3. **Leyenda Dinámica (Abajo a la izquierda):** Muestra el catálogo de terrenos del escenario activo con sus colores representativos y costos asociados, junto a los símbolos de inicio, fin y recurso.
4. **Panel Lateral de Control (Derecha):** Organizado mediante un cuaderno de pestañas (`Control` , `Resultados` , `Estadísticas` , `Comparación` , `Escenario`) para parametrizar las pruebas y analizar métricas.

6. Modos de Operación (Modo JSON vs. Modo Exploración)

En la esquina superior derecha encontrará dos botones de selección que cambian el comportamiento del programa:

A. Modo JSON (Ejecuta lo que dice el campo `prueba`)

- **Propósito:** Modo estricto de interoperabilidad diseñado para la evaluación docente.
- **Comportamiento:** Lee automáticamente el bloque `"prueba"` definido dentro del archivo JSON cargado.
- **Bloqueo de Controles:** Deshabilita los selectores manuales de unidad, coordenadas y algoritmo para garantizar que se ejecute con absoluta fidelidad lo especificado en el archivo externo.

B. Modo Exploración (Recomendado para el usuario)

- **Propósito:** Modo de análisis libre y experimentación interactiva.
- **Comportamiento:** Desbloquea todos los controles de la interfaz. Permite al usuario:
  - Cambiar libremente la unidad de origen.
  - Seleccionar la celda objetivo manualmente o **haciendo clic directamente sobre el tablero**.
  - Alternar entre algoritmos (BFS, DFS, UCS, A*, Beam Search) y heurísticas; al elegir `BEAM_SEARCH` aparece el campo `k` (ancho del haz).
  - Seleccionar el controlador de cada bando (Humano, Alfa-Beta, Minimax, Azar) y la profundidad de Minimax.
  - Evaluar estados arbitrarios con Minimax o Alfa-Beta y compararlos lado a lado.

7. Guía de Uso Paso a Paso

7.1 Carga y Validación de Escenarios JSON

1. Haga clic en el botón **`Cargar JSON…`** de la barra superior.
2. Navegue en el cuadro de diálogo y seleccione un archivo con formato `.json` (por ejemplo, dentro de `escenarios/casos minimos/`).
3. El sistema someterá el archivo a una **validación rigurosa**:
   - **Si el archivo es válido:** El tablero se actualizará inmediatamente, la leyenda adaptará sus colores y costos al JSON cargado, y se desplegará el mensaje ✓ `Escenario válido`.
   - **Si el archivo es inválido (regla de todo o nada):** El escenario previo **no se modificará**. Se abrirá una alerta de error y la pestaña `Escenario` detallará con precisión cada inconsistencia encontrada (muros sobrepasados, coordenadas fuera de límites, costos negativos, etc.).

7.2 Misión 1: Búsqueda de Rutas (Navegación Unipersonal)

Permite resolver problemas de navegación desde la posición de una unidad hasta una coordenada objetivo, ignorando temporalmente la presencia de adversarios.

```
+-------------------------------------------------------------+
| Parámetros de búsqueda                                      |
| Algoritmo:         [ A_ESTRELLA          v ]                |
| Unidad de inicio:  [ A1                  v ]                |
| Objetivo (f, c):   [ 10 ]  [ 16 ]  (o haz clic en el mapa)  |
| Heurística (A*):   [ propia              v ]                |
| Ancho del haz (k): [ 2 ]   (solo visible con BEAM_SEARCH)   |
| [X] Mostrar estados explorados                              |
|                                                             |
| [ Ejecutar búsqueda ] [ Comparar todos ] [ Verificar camino]|
+-------------------------------------------------------------+
```

Pasos para ejecutar:

1. En la pestaña **Control**, pulse el botón **`Búsqueda de ruta`** .
2. Seleccione el **Algoritmo** deseado en la lista desplegable:
   - `BFS` : Ruta con el menor número de saltos ortogonales (ignora costos de terreno).
   - `DFS` : Exploración en profundidad (camino no óptimo, útil para observar ramas).
   - `UCS` : Ruta con el menor costo acumulado $g(n)$ considerando el terreno.
   - `A_ESTRELLA` : Búsqueda informada $f(n) = g(n) + h(n)$.
   - `BEAM_SEARCH` : Búsqueda en haz con memoria acotada; solo expande los k mejores estados de cada nivel.
3. Si seleccionó `A_ESTRELLA` , elija la **Heurística**:
   - `manhattan` : Distancia Manhattan multiplicada por el costo mínimo del mapa (admisible).
   - `propia` : Heurística informada por el terreno de la meta (domina a Manhattan y expande menos nodos).

   > **Note**
   >
   > El código incluye también una tercera heurística, `euclidiana` , pero **no aparece en el desplegable de la interfaz** (es menos informada que Manhattan en cuadrículas ortogonales).

4. Si seleccionó `BEAM_SEARCH` , ajuste el parámetro **`k`** (ancho del haz). Un `k` pequeño acelera la búsqueda pero puede quedar atrapado sin solución; un `k` grande se aproxima a A*.
5. Indique la **Unidad de inicio** y la coordenada **Objetivo**.

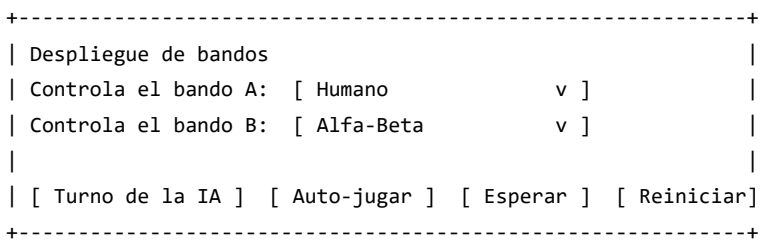
   > **Tip**
   >
   > En *Modo Exploración*, simplemente haga **clic izquierdo sobre una celda transitable del tablero** para fijarla como objetivo instantáneamente.

6. Presione **`Ejecutar búsqueda`** :
   - El tablero trazará la ruta en color cian y mostrará los nodos explorados con puntos ámbar.
   - La aplicación cambiará automáticamente a la pestaña **Resultados** para mostrar el tiempo en milisegundos, longitud, costo y nodos expandidos.
7. Presione **`Verificar camino`** :
   - Ejecuta el motor formal de verificación para comprobar que el camino no atraviese obstáculos, no contenga saltos diagonales y que el costo coincida al 100% con la suma real de los terrenos pisados. Si el algoritmo no encontró ruta, la columna de verificación muestra `–` (no hay nada que validar).
8. Presione **`Comparar todos`** :

- Ejecuta todos los algoritmos disponibles (BFS, DFS, UCS, A* y Beam Search) sobre el mismo problema de forma consecutiva y abre la pestaña **Comparación** para ver quién fue el más rápido, el más barato y el de menor expansión.

7.3 Misión 2: Partida Táctica por Turnos (Humano vs. IA / IA vs. IA)

Permite simular el juego completo bajo las reglas deterministas de TacticalGrid.



Configuración de Controladores:

Cada bando (`A` y `B`) puede ser asignado a:

- **`Humano`** : Control manual interactivo mediante clics.
- **`Alfa-Beta`** : Inteligencia Artificial guiada por Minimax con poda α-β.
- **`Minimax`** : Inteligencia Artificial guiada por árbol Minimax exhaustivo.
- **`Azar (provisional)`** : Realiza movimientos aleatorios legales.

> **Note**
>
> Los controladores automático usan la **profundidad** configurada en la tarea `Decisión adversarial` (por defecto 3). `Alfa-Beta` ordena las acciones con la variante `heurístico` y `Minimax` con el orden `estandar`. Si el árbol no devuelve ninguna acción (o el estado ya es terminal), se elige una acción aleatoria legal o se detiene, respectivamente.

Cómo Jugar en Modo Humano:

1. En el tablero, haga **clic sobre una unidad de su bando** (en el turno correspondiente). La unidad quedará seleccionada.
2. El tablero dibujará un **recuadro punteado** sobre las celdas legales a las que puede moverse.
3. Haga **clic sobre la casilla destino** para efectuar el desplazamiento.
4. Si prefiere que la unidad no se mueva y conserve su posición, presione el botón **`Esperar con la unidad elegida`** .

Reglas Especiales Visibles Durante la Partida:

- **Recogida del recurso:** Al pisar la celda donde está el recurso, la unidad lo toma automáticamente.
- **Interceptación táctica (Regla R4):** Si una unidad enemiga entra a la casilla del portador rival, **el portador es eliminado de la partida**, el recurso cae en esa casilla y quien interceptó lo toma de inmediato.
- **Victoria:** Ocurre cuando una unidad que transporta el recurso pisa la casilla de su propia base, o cuando el adversario se queda sin unidades vivas.

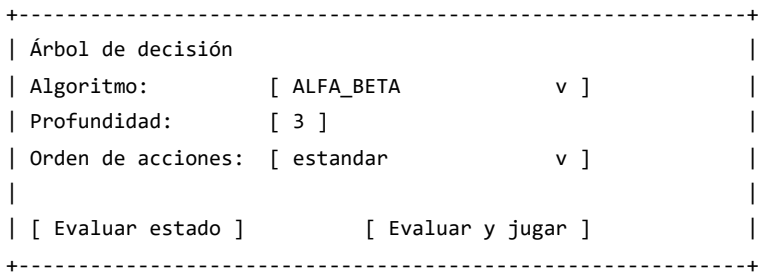
Modos Automatizados:

- **`Turno de la IA`** : Si le corresponde jugar a un bando controlado por IA, calcula y aplica su jugada óptima.

- **`Auto-jugar`** : Inicia un bucle continuo donde las IAs compiten turno a turno en tiempo real hasta que una logre la victoria o se alcance el límite de seguridad.

7.4 Misión 3: Toma de Decisiones Adversarial (Minimax y Alfa-Beta)

Diseñado para analizar en profundidad cómo razona la IA frente a un adversario, evaluar tableros concretos y medir el impacto de la poda.



Parámetros:

1. **Algoritmo:**
   - `MINIMAX` : Búsqueda adversarial exhaustiva hasta la profundidad límite.
   - `ALFA_BETA` : Búsqueda adversarial con poda de ramas irrelevantes (α-β).
2. **Profundidad Máxima:** Número de capas o plies a explorar en el árbol (1 a 8).
3. **Orden de Acciones:**
   - `estandar` : Orden determinista nativo del generador de sucesores.
   - `inverso` : Invierte el orden de los sucesores (útil para evidenciar peor caso de poda).
   - `heuristico` : Ordena las mejores acciones primero (capturas e intercepciones al inicio), maximizando la poda alfa-beta.

Botones de Acción:

- **`Evaluar estado`** : Calcula el valor Minimax del tablero actual, la jugada recomendada, los nodos evaluados y podados, y resalta en el mapa la casilla hacia donde la IA aconseja mover, **sin alterar el estado de la partida**.
- **`Evaluar y jugar`** : Calcula la mejor jugada y la ejecuta inmediatamente sobre el tablero, avanzando el turno.
- Si el estado ya es terminal (hay ganador), ambos botones muestran el aviso `Fin de partida` y no ejecutan el algoritmo.

Comparar Minimax contra Alfa-Beta:

1. Estando en la tarea **`Decisión adversarial`** , abra la pestaña **Comparación** y pulse **`Comparar todos`** .
2. La tabla mostrará dos filas (`MINIMAX` y `ALFA_BETA`) con la misma profundidad y el mismo orden de acciones, e indicará si ambos algoritmos eligieron la **misma acción**, sus valores y cuántos nodos evaluó y podó cada uno.
3. Haga clic en una fila para resaltar la acción recomendada sobre el tablero.

8. Interpretación de Pestañas y Paneles

8.1 Panel de Resultados y Verificación (`Resultados`)

Muestra la ficha técnica completa de la última búsqueda o evaluación adversarial ejecutada:

- **Métricas Principales:** Algoritmo, Éxito, Longitud del camino, Costo acumulado, Nodos generados, Nodos expandidos, Máximo de frontera y Tiempo de ejecución en milisegundos.
- **En una evaluación adversarial** la misma tabla muestra: Algoritmo, Turno analizado, Acción recomendada, Valor obtenido, Profundidad explorada, Nodos generados, Nodos evaluados (hojas), Nodos podados, Tiempo (ms) y Orden de acciones.
- **Detalle del Camino:** Lista ordenada de coordenadas `(fila, columna)` desde el origen hasta el objetivo.
- **Diagnóstico de Verificación:** Lista de 6 comprobaciones automáticas con indicador de éxito (✓) o fallo (✗):
  i. Inicio en la celda correcta.
  ii. Todas las celdas dentro de los límites.
  iii. Ninguna celda es obstáculo.
  iv. Pasos contiguos y ortogonales (sin diagonales).
  v. Costo reportado idéntico a la suma real de costos del mapa.
  vi. Llegada a la meta.
- **Salida JSON:** Estructura JSON estandarizada lista para ser copiada o documentada en informes.

8.2 Panel de Estadísticas y Comparación de Heurísticas (`Estadísticas`)

- **Resumen por Algoritmo:** Tabla con los promedios históricos de costo, expansiones y tiempos de cada algoritmo ejecutado durante la sesión.
- **Historial de Ejecuciones:** Bitácora detallada de cada prueba corrida.
- **Gráfico de Barras Interactivo:** Canvas dinámico que grafica la métrica elegida (Tiempo, Nodos Expandidos, Costo, etc.).
- **Comparador de Heurísticas (Abajo):** Permite seleccionar un algoritmo (ej. `A_ESTRELLA`) y comparar sus métricas evaluando simultáneamente las heurísticas **Manhattan y Propia** sobre el mismo problema (solo aparece cuando el algoritmo activo es `A_ESTRELLA`).
- **Exportar CSV:** Permite guardar todo el historial experimental en un archivo `.csv` compatible con Excel.

8.3 Panel de Comparación de Algoritmos (`Comparación`)

Se activa al presionar el botón **`Comparar todos`** . Según la tarea activa:

- **Tarea `Búsqueda de ruta` :** tabla con BFS, DFS, UCS, A* y **Beam Search** lado a lado.
- **Tarea `Decisión adversarial` :** tabla con **MINIMAX** y **ALFA_BETA** lado a lado (misma profundidad y orden de acciones), con columna de acción, valor, profundidad, nodos generados/evaluados/podados y tiempo. Al seleccionar una fila se resalta la acción en el tablero.
- En ambos modos se indica quién logró el **menor costo**, el **camino más corto**, el **menor número de nodos expandidos** y la **mayor velocidad**.
- **Interactividad en el mapa:** Al hacer clic sobre cualquier fila de la tabla, el tablero táctico cambiará inmediatamente para dibujar el camino y los nodos explorados por ese algoritmo específico, permitiendo contrastar visualmente las rutas.

8.4 Panel de Escenario y Registro de Eventos (`Escenario`)

- **Ficha del Escenario:** Dimensiones de la cuadrícula, catálogo de terrenos y costos, ubicación de bases, posición del recurso, listado de unidades activas y configuración de prueba del archivo JSON.
- **Registro de Eventos (Log):** Bitácora textual donde se imprimen cronológicamente todas las acciones ocurridas: jugadas realizadas, cambios de portador del recurso, advertencias y victorias.

9. Guía para Reproducir los Casos Mínimos de Evaluación

A continuación se indica cómo cargar y demostrar ante el profesor cada uno de los casos exigidos en la rúbrica:

Caso	Archivo JSON (`escenarios/casos minimos/`)	Operación en la Interfaz	Qué debe observarse
Caso 1 BFS vs DFS	`caso01_bfs_vs_dfs.json`	Cargar JSON → Pulsar `Comparar todos` .	Ambos encuentran camino. BFS halla la ruta más corta (menos movimientos); DFS recorre un camino largo y sinuoso.
Caso 2 Ruta corta vs Económica	`caso2_ruta_corta_vs_economica.json`	Cargar JSON → Pulsar `Comparar todos` .	**BFS** toma 6 pasos atravesando pantano (costo 60). **UCS** toma 10 pasos rodeando por camino de costo 1 (costo total 19). Menos pasos resulta más costoso.
Caso 3 Obstáculo y heurística	`caso3_obstaculo_y_heuristica.json`	Cargar JSON → Ejecutar `A_ESTRELLA` con Manhattan → Marcar `Mostrar explorados` .	La meta está cerca en línea recta, pero una trampa en "U" engaña a la

Caso	Archivo JSON (`escenarios/casos minimos/`)	Operación en la Interfaz	Qué debe observarse
			heurística, obligándola a explorar el fondo antes de rodear.
Caso 4 Comparación de heurísticas	`caso4_comparacion_heuristicas.json`	Ir a pestaña `Estadísticas` → Elegir `A_ESTRELLA` → Pulsar `Comparar heurísticas` .	Ambas hallan el costo óptimo idéntico (52, longitud 22), pero la **Heurística Propia** expande menos nodos (232) que Manhattan (353) gracias a su dominancia; también es más rápida.
Caso 5 Beam Search	`caso05_beam_search.json`	Cargar JSON → Probar variando el parámetro k en $\{1, 2, 4, 8\}$.	Con $k = 1$ y $k = 2$ el haz queda atrapado y **no encuentra ruta**; con $k = 4$ sí llega pero con camino subóptimo (costo 55); con $k = 8$ alcanza el **óptimo 31**, el mismo que A* y UCS.
Caso 6 Estado compuesto	`caso06a_sin_recurso.json` `caso06b_con_recurso.json`	Ejecutar script: `python experimentos/caso06_estado_compuesto.py`	La misma posición física $(10, 10)$

Caso	Archivo JSON (`escenarios/casos minimos/`)	Operación en la Interfaz	Qué debe observarse
			produce estados y hashes distintos porque en 6b la unidad lleva el recurso, habilitando acciones de intercepción.
Caso 7 Decisión adversarial	`caso07_decision_adversarial.json`	Cargar JSON → Tarea `Decisión adversarial` → Pulsar `Evaluar estado` .	El rival B1 porta el recurso. Minimax recomienda mover a la casilla de B1 para **interceptarlo y eliminarlo**, en vez de correr ciegamente.
Caso 8 Minimax vs Alfa-Beta	`caso08_minimax_vs_alfa_beta.json`	Evaluar con `MINIMAX` y luego con `ALFA_BETA` (misma prof. 3).	Ambos eligen **exactamente la misma jugada y el mismo valor**, pero Alfa-Beta evalúa un 83% menos de nodos gracias a las podas.
Caso 9 Orden de acciones	`caso09_orden_acciones.json`	Evaluar con `ALFA_BETA` alternando `inverso` vs `heuristico` .	El orden heurístico (mejores acciones primero) reduce los nodos

Caso	Archivo JSON (`escenarios/casos minimos/`)	Operación en la Interfaz	Qué debe observarse
			evaluados de 1,829 a solo 231 nodos.
Caso 10 Profundidad Minimax	`caso10_profundidad.json`	Evaluar con profundidades 1, 2, 3 y 4.	Los nodos evaluados crecen de 15 a 433 y el valor Minimax alterna (5.00, 0.00, 5.00, 0.00): cada capa extra anticipa mejor la respuesta rival a cambio de un crecimiento de tiempo medido.

Tip

Para ejecutar y visualizar automáticamente en consola un reporte consolidado de los Casos 7, 8, 9 y 10:

```
python experimentos/ejecutar_casos_adversariales.py
```

El reporte completo (casos 1–5, robustos `R01-R08` y adversariales 7–10) se regenera con:

```
python experimentos/ejecutar_experimentos_busqueda.py
```

y queda guardado en `documentacion/resultados_experimentos_busqueda.md` .

10. Escenarios Robustos de Evaluación

Los escenarios robustos se encuentran en la carpeta `escenarios/casos_robustos/` . Todos son JSON **válidos** y su propósito es someter a la aplicación a situaciones límite: mapas no cuadrados, metas inalcanzables, unidades encerradas, costos fraccionarios, dimensiones y nombres de terreno nuevos, tableros grandes y estados iniciales ya resueltos.

Para utilizarlos, cargue el archivo con **`Cargar JSON…`** y observe el comportamiento. Los escenarios con `prueba.modo = "busqueda"` ejecutan la búsqueda automáticamente en *Modo JSON*; los de `prueba.modo = "partida"` cargan directamente la partida.

Archivo (`escenarios/casos_robustos/`)	Contenido	Qué debe observarse
`r01_meta_inalcanzable.json`	Mapa 20×20; meta `(15,15)` rodeada por muros	Ningún algoritmo encuentra camino ni se cuelga: `Resultados` muestra **sin éxito**.
`r02_inicio_igual_meta.json`	A1 inicia en `(5,5)` y la meta es `(5,5)`	Éxito inmediato con **longitud 0** y **costo 0**.
`r03_corredor_1x20.json`	Mapa no cuadrado de 1×20 (camino y pasto)	Un único camino de **19 movimientos** y costo **29**; todos los algoritmos coinciden.
`r04_unidad_encerrada.json`	A1 en `(5,5)` rodeada de muros	Las búsquedas fallan. En partida, A1 solo puede **ESPERAR** (11 acciones legales para el bando A).
`r05_objetivo_en_muro.json`	La prueba apunta a `(10,10)` , que es un muro	La búsqueda se **rechaza** con un mensaje claro antes de ejecutar el algoritmo.
`r06_costos_fraccionarios.json`	Autopista de costo `0.5` en la fila 2	Camino de **21 movimientos** y costo **11.5**; A* con Manhattan iguala el costo de UCS.
`r07_dimensiones_y_nombres_nuevos.json`	Mapa 8×12 con terrenos `sendero` , `arena` , `agua` y `roca` ; 4 unidades por bando y campos extra	El escenario **carga sin errores**; los costos se toman del JSON y UCS y A* coinciden en costo.
`r08_mapa_grande_40x40.json`	Mapa 40×40 con terreno aleatorio (semilla fija) y ~5 % de muros	Todos los algoritmos hallan solución en **menos de 5 s**; UCS y A* coinciden en costo.
`r09_portador_en_su_base.json`	A1 lleva el recurso y está sobre la base A	La partida ya está **ganada por el bando A** al cargar.
`r10_unidad_sobre_el_recurso.json`	A1 inicia sobre el recurso con `portador_recurso: null`	A1 aparece como **portadora** y todavía no hay ganador.

Note

Si alguno de estos archivos se muestra como inválido, se trata de un defecto de validación: estos escenarios fueron construidos para ser aceptados.

Los robustos `R01` a `R08` (los de modo búsqueda) se ejecutan automáticamente y sus métricas quedan registradas en `documentacion/resultados_experimentos_busqueda.md` al correr `python experimentos/ejecutar_experimentos_busqueda.py` . `R09` y `R10` se validan cargándolos en la interfaz.

11. Escenarios Inválidos (Rechazo Atómico)

La carpeta `escenarios/casos_invalidos/` contiene un archivo por cada regla de validación incumplida. El comportamiento esperado es siempre el mismo: **rechazo atómico**, es decir, el escenario anterior permanece cargado y sin modificaciones, se muestra una alerta y la pestaña `Escenario` enumera los errores detectados.

Archivo (`escenarios/casos_invalidos/`)	Regla que incumple	Fragmento esperado del error
`inv01_dimensiones_inconsistentes.json`	`mapa.filas` no coincide con el terreno	`mapa.filas es 5`
`inv02_terreno_inexistente.json`	El terreno usa un tipo no declarado	`no existe en tipos_terreno`
`inv03_costo_cero.json`	Un terreno transitable tiene costo `0`	`costo positivo`
`inv04_base_fuera_del_mapa.json`	La base A queda fuera de la cuadrícula	`bases.A est`
`inv05_unidad_sobre_muro.json`	Una unidad inicia sobre una celda no transitable	`no transitable`
`inv06_id_repetido.json`	Dos unidades comparten el mismo identificador	`repetido`
`inv07_bando_invalido.json`	Una unidad declara un bando distinto de A o B	`bando debe ser A o B`
`inv08_turno_invalido.json`	El turno inicial no es A ni B	`turno debe ser A o B`
`inv09_portador_inexistente.json`	El portador no corresponde a una unidad existente	`no es una unidad existente`
`inv10_json_mal_formado.json`	El archivo no es un JSON sintácticamente válido	`JSON mal formado`
`inv11_falta_campo_obligatorio.json`	Falta la sección `bases`	`Faltan campos obligatorios`
`inv12_recurso_fuera_del_mapa.json`	El recurso está fuera de los límites	`recurso est`

Important

Cargue estos archivos uno por uno y verifique que el tablero **no cambie**. Si el escenario anterior desaparece o se reemplaza, el mecanismo de rechazo atómico no está funcionando correctamente.

12. Preguntas Frecuentes y Solución de Problemas

1. ¿Por qué al hacer clic en el mapa no cambia la celda objetivo?

Verifique que esté activado el **Modo Exploración** en la barra superior. En *Modo JSON*, los controles se bloquean deliberadamente para respetar el contrato del archivo cargado.

2. ¿Qué significa el mensaje "Escenario rechazado (X errores)"?

El archivo JSON suministrado no cumple con las reglas mínimas del enunciado (por ejemplo: hay unidades sobre muros, coordenadas fuera de límites, o faltan campos obligatorios). TacticalGrid aplica rechazo atómico: el mapa anterior se mantiene intacto. Diríjase a la pestaña `Escenario` para leer la lista detallada de errores reportados.

3. ¿Cómo exportar los datos para el informe de experimentación?

En la pestaña `Estadísticas`, pulse el botón `Exportar CSV`. Esto guardará todas las corridas, métricas, algoritmos y parámetros en una hoja de cálculo que podrá abrir en Microsoft Excel o Google Sheets para generar gráficos.

4. ¿Por qué Minimax tarda mucho tiempo en profundidades mayores a 5?

El árbol de juego en mapas de 20×20 con varias unidades tiene un factor de ramificación alto ($b \approx 15$–25). Por ello, la complejidad temporal de Minimax es exponencial ($O(b^d)$). Para profundidades de 3 a 5 se recomienda utilizar **Alfa-Beta** con orden de acciones **heurístico**, el cual reduce significativamente el espacio de búsqueda.

5. ¿Beam Search no encuentra ruta con `k` pequeño, es un error?

No. Con un ancho de haz reducido la búsqueda solo conserva los k mejores estados de cada nivel y puede descartar la única ruta viable (por ejemplo, en el Caso 5 ni `k=1` ni `k=2` encuentran camino). Incrementa `k` (4 u 8) hasta recuperar una solución; con `k` suficientemente grande se alcanza el mismo costo óptimo que A*.

6. ¿El botón `Evaluar estado` responde "Fin de partida"?

El estado actual ya es terminal: existe un ganador (o bien el portador alcanzó su base, o bien un bando perdió todas sus unidades). Minimax y Alfa-Beta solo se ejecutan sobre estados no terminales; reinicia la partida para evaluar un nuevo árbol.

7. ¿La columna `Verif.` de la comparación muestra "—"?

Significa que ese algoritmo no encontró camino (o el objetivo era un muro, y por lo tanto la búsqueda se rechazó). Solo se verifica el camino cuando existe y llega a la meta; no es un fallo de verificación.

Fin del Manual de Usuario.